



Responsiveness + Positioning + Flex

Mahhek Tahir



Responsive Design.

- Grid System (Bootstrap, tailwind)
- CSS Media Queries
- Flexbox

The **viewport** is the visible area of the webpage inside the browser window.



Media Queries

- **What are Media Queries?**
 - CSS rules that apply styles based on device characteristics such as screen width, height, or orientation.
- **Common Use Cases:**
 - Adjusting layouts for different screen sizes.
 - Changing font sizes on smaller screens.
 - Hiding or showing elements.



```
@media (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}  
  
@media (min-width: 601px) and (max-width: 900px) {  
  body {  
    background-color: purple;  
  }  
}  
  
@media (orientation: portrait) {  
  img {  
    width: 100%;  
  }  
}
```



```
/* Extra small devices (phones, 600px and down) */  
@media only screen and (max-width: 600px) {...}  
  
/* Small devices (portrait tablets and large phones,  
600px and up) */  
@media only screen and (min-width: 600px) {...}  
  
/* Medium devices (landscape tablets, 768px and up) */  
@media only screen and (min-width: 768px) {...}  
  
/* Large devices (laptops/desktops, 992px and up) */  
@media only screen and (min-width: 992px) {...}  
  
/* Extra large devices (large laptops and desktops,  
1200px and up) */  
@media only screen and (min-width: 1200px) {...}
```



CSS Layout – Position

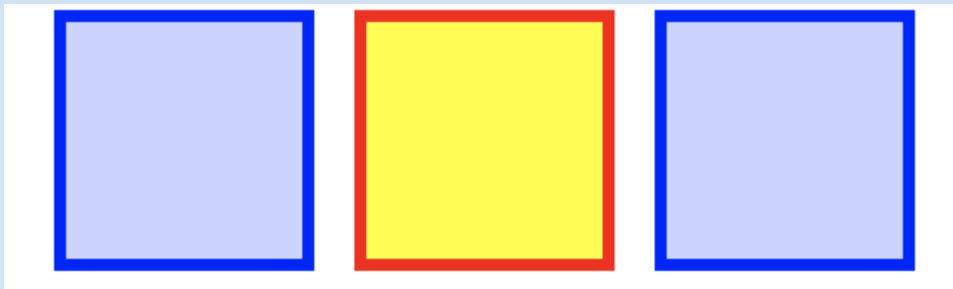
The position is CSS sets how an element is positioned in a document.

- ▶ **Static:** Default positioning; elements are placed in the normal flow. (default)
- ▶ **Relative:** Positioned relative to its normal position.
- ▶ **Absolute:** Positioned relative to the nearest positioned ancestor.
- ▶ **Fixed:** Positioned relative to the viewport; does not move on scroll.
- ▶ **Sticky:** Switches between relative and fixed based on scroll position.

Static



The element is positioned according to the **Normal Flow** of the document. The top, right, bottom, left, and **z-index** properties have *no effect*. This is the default value.

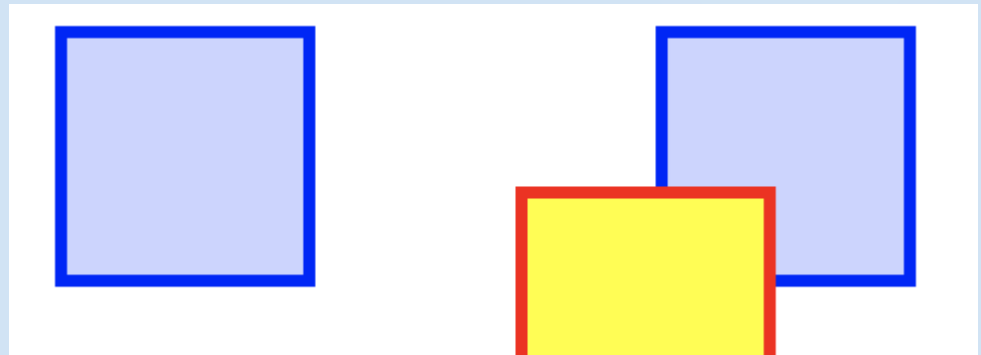


Relative



The element is positioned according to the normal flow of the document. and then offset *relative to itself* based on the values of `top`, `right`, `bottom`, and `left`. The offset does not affect the position of any other elements; thus, the space given for the element in the page layout is the same as if position were `static`.

```
position: relative;  
top: 40px; left: 40px;
```



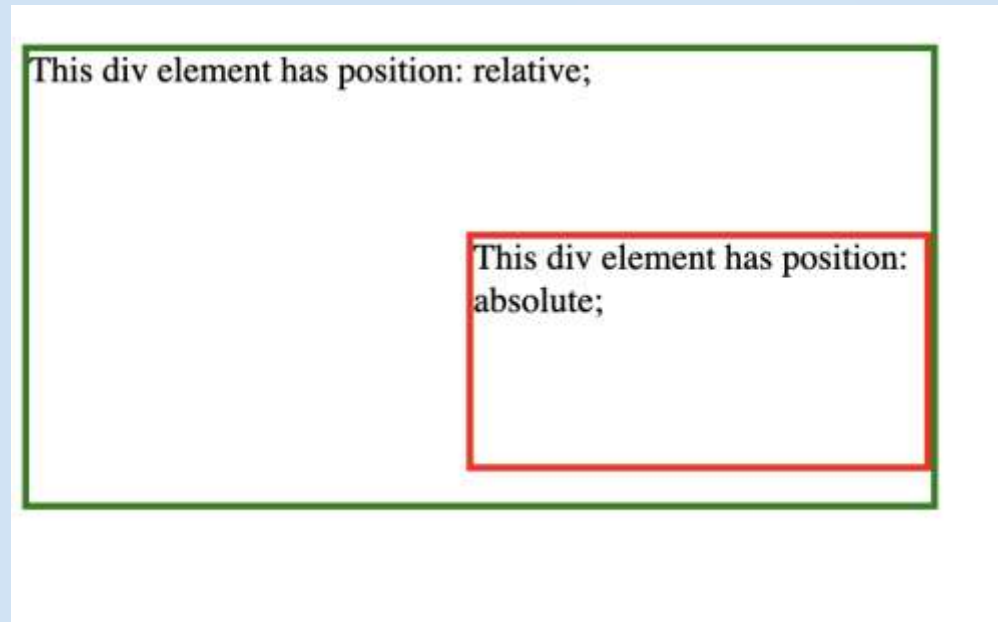


Absolute

An element with `position: absolute;` is positioned relative to the nearest positioned ancestor, Any ancestor with **position NOT equal to static**: If not ancestor is relative? The absolutely positioned element is positioned **relative to the initial containing block** (usually the **page**)

```
div.relative {  
  position: relative;  
  width: 400px;  
  height: 200px;  
  border: 3px solid green;  
}
```

```
div.absolute {  
  position: absolute;  
  top: 80px;  
  right: 0;  
  width: 200px;  
  height: 100px;  
  border: 3px solid red;  
}
```





Fixed & Sticky

- ▶ `position: fixed`: The element is removed from the normal document flow. Its final position is determined by the values of top, right, bottom, and left. For instance a website with a header and navigation that always stays visible when you are scrolling is a good example of fixed positioning.
- ▶ `position: sticky` basically acts like `position: relative` until an element is scrolled beyond a specific offset, in which case it turns into `position: fixed`,
- ▶ Before it sticks, its position is defined by its **normal position in the document flow**. `top: 20px`; effects after it sticks

https://www.w3schools.com/css/tryit.asp?filename=trycss_position_sticky

Using position: fixed;

An element with position: fixed; is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled:

```
div.fixed {  
  position: fixed;  
  bottom: 0;  
  right: 0;  
  width: 300px;  
  height: 200px;  
  border: 3px solid  
}
```

This div element has position: fixed;

Understanding Flexbox Layout



What is Flexbox?

- Aligns items in **row or column**

```
.container {  
    display: flex;  
}
```



```
<div class="container">
```

```
  <div class="card">
```

```
    <h3 class="card-title">Card One</h3>
```

```
    <p class="card-text">This is some sample content.</p>
```

```
    <button class="btn">Click Me</button>
```

```
  </div>
```

```
  <div class="card">
```

```
    <h3 class="card-title">Card Two</h3>
```

```
    <p class="card-text">This is some sample content.</p>
```

```
    <button class="btn">Click Me</button>
```

```
  </div>
```

```
  <div class="card">
```

```
    <h3 class="card-title">Card Three</h3>
```

```
    <p class="card-text">This is some sample content.</p>
```

```
    <button class="btn">Click Me</button>
```

```
  </div>
```

```
</div>
```



Applying Flexbox to Parent

- Flexbox is applied to the parent
- All direct children become flex items
- `.card` now behaves as flexible boxes
- Layout becomes horizontal by default

Flexbox always works on the parent.

```
.container {  
    display: flex;  
}
```

FLEXBOX: APPLIED TO THE PARENT

Flexbox always works on the **parent**.

1 FLEXBOX APPLIED TO PARENT

```
.container {  
  display: flex;  
}  
  
.card {  
  /* flex items (children) */  
}
```

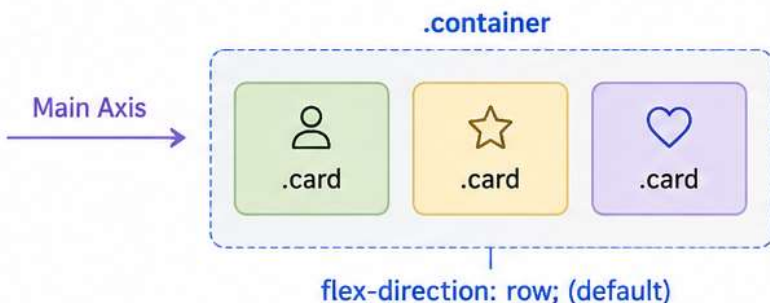


↑
`display: flex;` is applied here

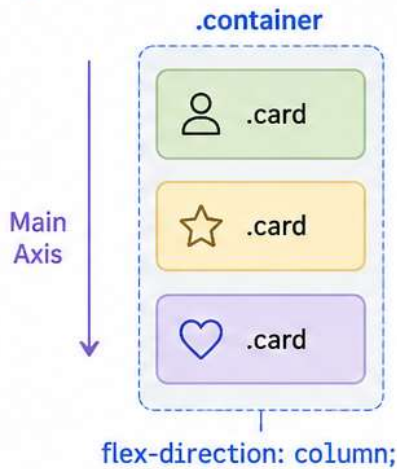
WHAT HAPPENS?

- All direct children become flex items
- `.card` now behaves as flexible boxes
- Layout becomes horizontal by default

2 ALIGN ITEMS IN ROW (DEFAULT)



3 ALIGN ITEMS IN COLUMN



KEY TAKEAWAY

- Add `display: flex;` to the parent.
- Direct children become flex items.
- Control direction with `flex-direction: row | column.`



Allowing Items to Move

- Default is `nowrap`
- `wrap` allows items to move to next line
- Prevents overflow
- Important for responsive layouts

Simple explanation:

Wrap allows items to break into a new row when space ends.

`gap` creates space between flex items

Cleaner than using margins

Works directly with Flexbox

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}
```


FLEXBOX: WRAP (**flex-wrap**)

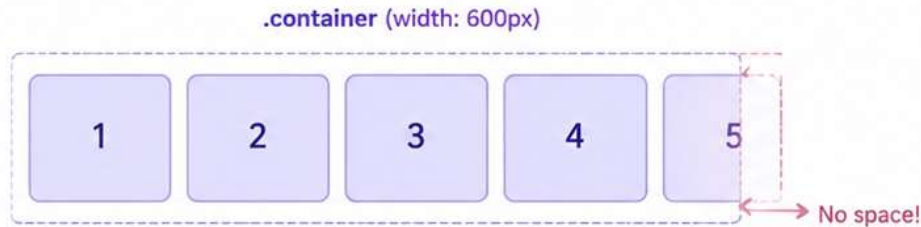


By default, items try to fit in a single row.

Use **flex-wrap** to allow items to move to the next line when there is not enough space.

1 DEFAULT: NO WRAP (**flex-wrap: nowrap**)

```
.container {  
  display: flex;  
  flex-wrap: nowrap;  
  /* default */  
}  
  
.card {  
  flex: 0 0 150px;  
  background: #e0e7ff;  
  padding: 1rem;  
  margin: 0.5rem;  
  border-radius: 8px;  
}
```



All items stay in one row and may shrink (or overflow) to fit.

FLEX-WRAP VALUES

nowrap
(default)

All items stay in a single line.

wrap

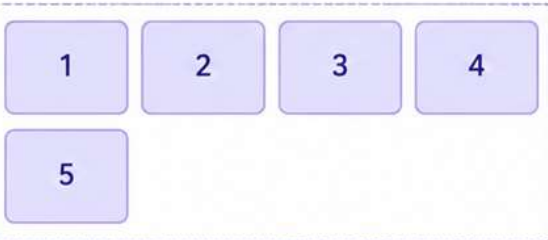
Items move to the next line when there is not enough space.

wrap-reverse

Same as wrap, but new lines are added above (reverse direction).

2 WRAP: ITEMS MOVE TO NEXT LINE (**flex-wrap: wrap**)

.container (width: 600px)

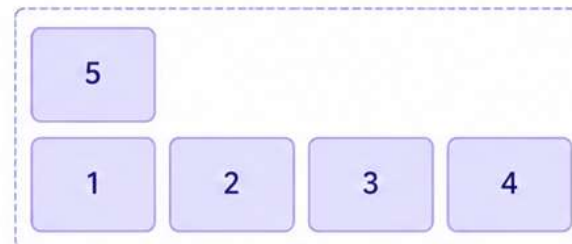


Items wrap to the next line when there is not enough space.

(Default direction: **row**)

3 WRAP-REVERSE: NEW LINES ABOVE (**flex-wrap: wrap-reverse**)

.container (width: 600px)



Items wrap to the next line, but new lines are added **above**.

(Reverse direction)



KEY TAKEAWAY: Use **flex-wrap: wrap**; to make your layout responsive and prevent overflow on smaller screens.

FLEXBOX: GAP



The **gap** property adds space between flex items. It works for both rows and columns.

</> EXAMPLE HTML

```
<div class="container">
  <div class="card">1</div>
  <div class="card">2</div>
  <div class="card">3</div>
  <div class="card">4</div>
</div>
```

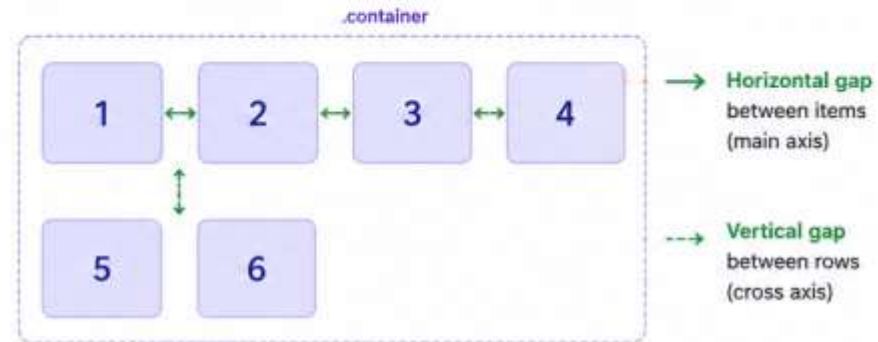
 = Flex Item (.card)

{ } CSS

```
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px; /* space between items */
}

.card {
  background: #d7dbff;
  padding: 20px 30px;
  border-radius: 8px;
  font-weight: bold;
  text-align: center;
  min-width: 100px;
}
```

🔗 RESULT (gap: 20px)



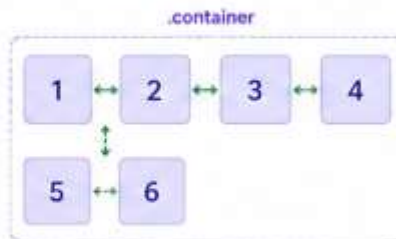
DIFFERENT GAP VALUES

gap: 0 (default)



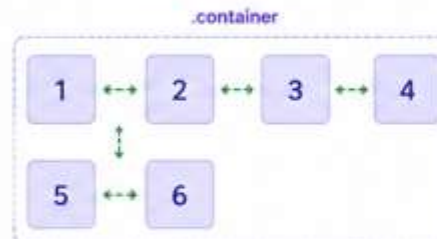
No space between items.

gap: 8px



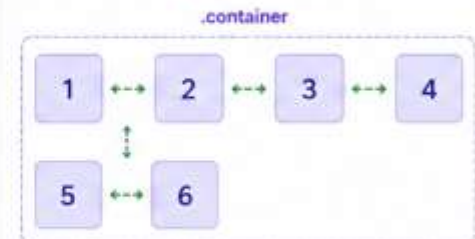
Small space between items.

gap: 20px



Medium space between items.

gap: 40px



Large space between items.



KEY TAKEAWAY: Use **gap** to control the space between flex items —

gap: <row-gap> <column-gap>; (e.g., **gap: 20px;** or **gap: 10px 30px;**)

→ First value → row gap (between rows)
↓ Second value → column gap (between items)

How It Becomes Responsive



We use **Media Queries** to change the layout at different screen widths.
Flexbox + flexible widths (% , `calc()`) make the cards adapt automatically.



MOBILE (< 768px)

One card per row (100% width)

```
.card {  
  flex: 1 1 100%;  
}
```

CSS

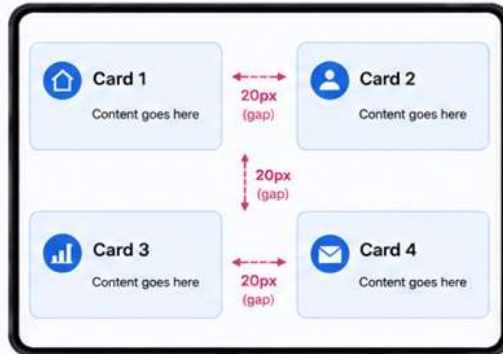


TABLET (≥ 768px)

Two cards per row (50% width - gap)

```
@media (min-width: 768px) {  
  .card {  
    flex: 1 1 calc(50% - 10px);  
  }  
}
```

10px is subtracted from each card



CONTAINER GAP

gap: 20px;

This is the original gap defined on the container.

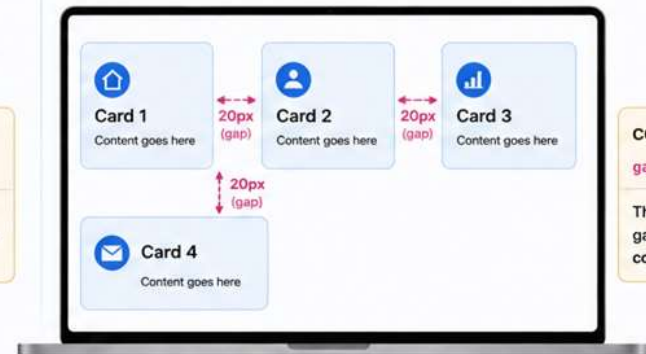


LAPTOP (≥ 1024px)

Three cards per row (33.33% width - gap)

```
@media (min-width: 1024px) {  
  .card {  
    flex: 1 1 calc(33.333% - 13.33px);  
  }  
}
```

13.33px is subtracted from each card



CONTAINER GAP

gap: 20px;

This is the original gap defined on the container.

✓ Takes full width
Mobile-first approach

Why `calc(50% - 10px)`?

Container gap is 20px.

In a row of 2 cards, there is 1 horizontal gap (20px) between them.
We subtract 20px from 100% and divide it across 2 cards.

Each card width

$$(50\% - 10\text{px}) + 20\text{px (gap)} + (50\% - 10\text{px}) = 100\%$$

10px is subtracted from each card

20px is the actual gap (original gap)

10px is subtracted from each card



So: $(50\% - 10\text{px}) + 20\text{px} + (50\% - 10\text{px}) = 100\%$
Both cards + the gap fit perfectly in 100% width.

Why `calc(33.333% - 13.33px)`?

Container gap is 20px.

In a row of 3 cards, there are 2 horizontal gaps (20px + 20px = 40px).
We subtract 40px from 100% and divide it across 3 cards.

Each card width

$$(33.333\% - 13.33\text{px}) + 20\text{px (gap)} + (33.333\% - 13.33\text{px}) + 20\text{px (gap)} + (33.333\% - 13.33\text{px}) = 100\%$$

13.33px is subtracted from each card

20px is the actual gap (original gap)

13.33px is subtracted from each card

20px is the actual gap (original gap)

13.33px is subtracted from each card



So: $(33.333\% - 13.33\text{px}) \times 3 + 20\text{px} + 20\text{px} = 100\%$
All three cards + the two gaps fit perfectly in 100% width.

Understanding: flex: 1 1 ...

The first two values control how items grow and shrink.

1 → flex-grow

How much the item can grow to take up extra space.

Example: Extra space is available



All items grow equally to fill the available space.

1 = Item can grow

0 = Item will not grow

Example CSS

```
.container {  
  display: flex;  
}  
  
.item {  
  flex: 1 1 auto; /* 1st value = 1 → grow */  
}
```

2 → flex-shrink

How much the item can shrink when there is not enough space.

Example: Not enough space (items must shrink)



All items shrink equally to fit in the smaller space.

1 = Item can shrink

0 = Item will not shrink

Example CSS

```
.container {  
  display: flex;  
}  
  
.item {  
  flex: 1 1 auto; /* 2nd value = 1 → shrink */  
}
```

Without flexbox nav menu?



Display block vs Display - inline

html

```
<ul>
  <li style="display:inline;">Home</li>
  <li style="display:inline;">About</li>
  <li style="display:inline;">Contact</li>
</ul>
```

Result:

nginx

Home About Contact



```
/* Basic Card Styling */
.card{
border:1px solid #eeee; /* Light gray border */
border-radius:10px; /* Rounded corners */
padding:20px; /* Internal spacing */
transition:transform 0.3s ease; /* Smooth hover effect */
}
/* Center Multiple Elements */
.card-title,.card-text{
text-align:center; /* Centers text */
}
/* First Card */
.card:first-child{
border-color:black; /* Styles only first card */
}
/* Last Card */
.card:last-child{
border-color:blue; /* Styles only last card */
}
/* Hover Effect */
.card:hover{
transform:scale(1.25); /* Enlarges card on hover */
}
/* Disabled Button */
button:disabled{
cursor:not-allowed; /* Disabled cursor */
opacity:0.6; /* Faded appearance */
}
```

Data in html



`data-*` attributes let us store **custom information** directly on an HTML element.

`data-id`, `data-role`, `data-price`, etc. are examples.

The browser doesn't give these attributes a built-in meaning.

They are mainly useful for **JavaScript to read and use information about an element**.

`data-name="value"`

```
<div data-id="25"></div>
```

```
<div data-category="books"></div>
```

```
<div data-price="1500"></div>
```

Data in html



```
<section class="item-list">

  <div class="item-box" data-item-id="501">
    <h4 class="item-name">Laptop Stand</h4>
    <span class="item-cost">$45.00</span>
    <button class="select-btn">Select</button>
  </div>

  <div class="item-box" data-item-id="502">
    <h4 class="item-name">USB Hub</h4>
    <span class="item-cost">$25.00</span>
    <button class="select-btn">Select</button>
  </div>

</section>

<p id="output"></p>
```


Resources to Learn



Slide: Resources to Learn More

FreeCodeCamp, Codecademy, or W3Schools.